

Sensors & Systems

Kalman Filter for Nonlinear Systems — EKF & UKF

Torben Knudsen | Electronic Systems, 8th Semester

Section 1 Why the Standard KF Breaks for Nonlinear Systems

The standard Kalman filter from Lecture 3 is the optimal estimator for *linear* systems. Most real physical systems, however, are **nonlinear** to some degree — a pendulum, a robot arm, a drone, a chemical reactor. This raises three important questions:

- Can states still be estimated in nonlinear systems?
- Does an *optimal* estimator exist?
- Does a *good* (not necessarily optimal) estimator exist?

The short answers: yes, sometimes, and yes. An optimal closed-form estimator generally does *not* exist for nonlinear systems — but practical approximations (EKF and UKF) work well in most cases.

What does “nonlinear” actually mean here?

In the linear KF the state equation and measurement equation use *matrices*:

$$x_{k+1} = \Phi_k x_k + \Gamma_k u_k + w_k, \quad y_k = H_k x_k + D_k u_k + v_k$$

A matrix multiplication $\Phi_k x_k$ is **linear**: the output scales proportionally with the input. Formally, a function f is linear if:

- Double the input $\Rightarrow$ double the output: $f(2x) = 2f(x)$
- Add two inputs $\Rightarrow$ add the outputs: $f(x + y) = f(x) + f(y)$

A matrix satisfies both — it only stretches, rotates, or flips a vector, never bends or curves it. This means the KF can compute exact means and covariances because everything stays predictable and proportional.

In a nonlinear system the matrices Φ_k and H_k are replaced by arbitrary **functions** f and h :

$$x_{k+1} = f(x_k, u_k) + w_k, \quad y_k = h(x_k, u_k) + v_k$$

f and h can be anything — sin, cos, products of states, square roots. A nonlinear function breaks the linearity rules: $\sin(2x) \neq 2\sin(x)$, so the output no longer scales predictably. The noise assumptions are unchanged (w_k and v_k white and uncorrelated), but the functions themselves can bend and curve.

Concrete examples of nonlinear f and h .

- **Pendulum**: the state is angle θ and angular velocity $\dot{\theta}$. The state equation contains $\sin \theta$ — nonlinear in the state.

- **Range sensor:** you measure the distance to an object whose position (p_x, p_y) is the state. $h = \sqrt{(p_x - s_x)^2 + (p_y - s_y)^2}$ — a square root of states, nonlinear.
- **IMU attitude** (from Lecture 5): converting gyro rates to orientation involves sin and cos of the current roll/pitch/yaw angles — nonlinear.

The NL DD estimation problem

The discrete-discrete nonlinear problem has the same structure as the linear DD problem — it just uses f and h instead of matrices:

$$\begin{aligned}x_{k+1} &= f(x_k, u_k) + w_k, & w_k &\sim \text{NID}(0, Q_k) \\y_k &= h(x_k, u_k) + v_k, & v_k &\sim \text{NID}(0, R_k) \\E[w_k v_l^\top] &= 0\end{aligned}$$

Symbol	Name	Meaning
x_k	State vector	What we want to estimate (position, velocity, angle, ...)
$f(\cdot, \cdot)$	State function	Nonlinear function describing how the state evolves; replaces Φ_k from the linear KF
u_k	Input/control	Known input applied at step k (force, voltage, ...)
w_k	Process noise	Random disturbances; $w_k \sim \text{NID}(0, Q_k)$
Q_k	Process noise cov.	Covariance of w_k — how uncertain the state equation is
y_k	Measurement vector	What the sensor actually outputs at step k
$h(\cdot, \cdot)$	Measurement function	Nonlinear function mapping state to sensor output; replaces H_k from the linear KF
v_k	Measurement noise	Sensor noise; $v_k \sim \text{NID}(0, R_k)$
R_k	Meas. noise cov.	Covariance of v_k — how noisy the sensor is

Given all measurements up to step k :

$$\mathcal{Y}_k \triangleq y_0, y_1, \dots, y_k, \quad u_0, u_1, \dots, u_k, \quad x_0 \sim \mathcal{N}(\hat{x}_0, P_0)$$

$\mathcal{Y}_k$ is everything the filter is **allowed to use** — its full information set:

- $y_0, y_1, \dots, y_k$ — every sensor reading collected so far.

- $u_0, u_1, \dots, u_k$ — every input/control signal applied (known exactly, because *you* applied them).
- $x_0 \sim \mathcal{N}(\hat{x}_0, P_0)$ — the initial guess for the starting state: modelled as Gaussian with mean $\hat{x}_0$ and covariance P_0 .

The hat $\hat{\cdot}$ always denotes an **estimate**: $\hat{x}_k$ is our best guess for the true state x_k .

Find $\hat{x}_k$ that minimises the mean square error:

$$E[(x_k - \hat{x}_k)^\top M (x_k - \hat{x}_k)], \quad M \geq 0$$

What is M ?

M is a **weighting matrix** — it controls how much the cost function penalises errors in each component of the state. The expression $(x_k - \hat{x}_k)^\top M (x_k - \hat{x}_k)$ is a weighted sum of squared errors between the true state and the estimate.

The key insight: the condition is only $M \geq 0$ (any valid weighting). This means **no matter how you weight the errors, the same estimator wins** — the conditional mean $E[x_k | \mathcal{Y}_k]$. You cannot do better in any direction simultaneously, regardless of whether you care more about position error, velocity error, or any combination. This is the theoretical justification for why the KF (which tracks the conditional mean) is the optimal estimator for linear Gaussian systems.

	Linear DD (Lecture 3)	Nonlinear DD
State eq.	$x_{k+1} = \Phi_k x_k + \Gamma_k u_k + w_k$	$x_{k+1} = f(x_k, u_k) + w_k$
Meas. eq.	$y_k = H_k x_k + D_k u_k + v_k$	$y_k = h(x_k, u_k) + v_k$
Optimal estimator	Kalman filter — exact, closed form	No closed-form solution in general
Practical estimator	KF	EKF or UKF

The NL CD estimation problem

The continuous-discrete nonlinear problem follows the same extension: replace the linear continuous model with a nonlinear SDE.

$$dx(t) = f(x(t), u(t)) dt + d\omega(t), \quad \omega(t) \sim \mathcal{W}(Q(t))$$

$$y(t_k) = h(x(t_k), u(t_k)) + v_k, \quad v_k \sim \text{NID}(0, R_k)$$

Symbol	Name	Meaning
$x(t)$	State vector	Continuously evolving state (angle, concentration, ...)
$f(\cdot, \cdot)$	Drift function	Nonlinear function for how the state changes continuously; replaces $F(t)x + B(t)u$ from the linear CD model
$u(t)$	Continuous input	Known control signal varying continuously in time
$d\omega(t)$	Wiener increment	Continuous process noise; infinitesimal random kick at each instant
$Q(t)$	Spectral density	Intensity of the continuous noise $d\omega(t)$
$y(t_k)$	Measurement	Discrete sensor output, sampled at times $t_0, t_1, \dots$
$h(\cdot, \cdot)$	Meas. function	Nonlinear mapping from state to sensor reading; replaces H_k
v_k	Measurement noise	Discrete sensor noise; $v_k \sim \text{NID}(0, R_k)$
R_k	Meas. noise cov.	Covariance of v_k — how noisy the sensor is

The continuous state evolves according to a *nonlinear* SDE between measurement times. The measurement at each discrete sample t_k is a nonlinear function of the current state.

Why no optimal estimator exists for nonlinear systems.

In the linear case, the conditional distribution $p(x_k | \mathcal{Y}_k)$ is always Gaussian, and its mean and covariance are all you need — the KF tracks them exactly.

For a nonlinear system, passing a Gaussian through a nonlinear function f produces a distribution that is *no longer Gaussian* — it is bent, skewed, multi-modal. Tracking the full shape of that distribution exactly requires infinitely many parameters.

Both EKF and UKF sidestep this by forcing the distribution to stay Gaussian (approximating it by a mean and covariance), just computed differently:

- **EKF**: approximate f as linear (tangent line at the current estimate) then propagate the Gaussian through that linear approximation.
- **UKF**: keep f exact but approximate the output Gaussian using a small set of carefully chosen sample points (sigma points).

Key takeaways — Section 1

- Nonlinear means the state/measurement functions are $f(x, u)$ and $h(x, u)$ — arbitrary functions, not matrices.

- The noise assumptions are unchanged: white, Gaussian, uncorrelated.
- No optimal closed-form estimator exists for nonlinear systems — passing a Gaussian through a nonlinear function destroys its Gaussian shape.
- EKF and UKF are both approximations that keep the distribution Gaussian, just with different strategies for computing the mean and covariance after the nonlinear step.

Compare the linear and nonlinear forms side by side:

	Linear DD	Linear CD
State eq.	$x_{k+1} = \Phi_k x_k + \Gamma_k u_k + w_k$	$dx(t) = F(t)x(t) dt + B(t)u(t) dt + d\omega(t)$
Meas. eq.	$y_k = H_k x_k + D_k u_k + v_k$	$y(t_k) = H_k x(t_k) + D_k u(t_k) + v_k$
State noise	$w_k \sim \text{NID}(0, Q_k)$	$\omega(t) \sim \mathcal{W}(Q(t))$
Meas. noise	$v_k \sim \text{NID}(0, R_k)$	$v_k \sim \text{NID}(0, R_k)$
	NL DD	NL CD
State eq.	$x_{k+1} = f(x_k, u_k) + w_k$	$dx(t) = f(x(t), u(t)) dt + d\omega(t)$
Meas. eq.	$y_k = h(x_k, u_k) + v_k$	$y(t_k) = h(x(t_k), u(t_k)) + v_k$
State noise	$w_k \sim \text{NID}(0, Q_k)$	$\omega(t) \sim \mathcal{W}(Q(t))$
Meas. noise	$v_k \sim \text{NID}(0, R_k)$	$v_k \sim \text{NID}(0, R_k)$

Section 2 The Extended Kalman Filter (EKF)

Basic idea

The EKF keeps the same two-step structure as the standard KF (time update + measurement update), but handles the nonlinear functions f and h by **linearising** them around the current best estimate before applying the KF equations. Four key points from the slides:

- The optimal estimator $E(x_k | \mathcal{Y}_k)$ cannot normally be obtained for nonlinear systems.
- The KF framework *can* still be used if we first linearise the nonlinear system.
- Linearisation can be done with a **fixed** point (chosen once) or a **variable** point (updated every step).
- The best EKF uses the **most recent state estimate** as the linearisation point and

keeps the nonlinear functions f and h wherever they appear directly (not just their linearised versions).

Linearisation: the Jacobian matrices

To linearise f and h with respect to the state x , we take their **partial derivatives** — these are called Jacobian matrices:

$$F_k(x) \triangleq \frac{\partial f(x)}{\partial x^\top}, \quad H_k(x) \triangleq \frac{\partial h(x)}{\partial x^\top}$$

Symbol	Name	Meaning
$F_k(x)$	State Jacobian	$n \times n$ matrix of partial derivatives of f ; replaces Φ_k in the covariance equations
$H_k(x)$	Output Jacobian	$p \times n$ matrix of partial derivatives of h ; replaces H_k in the gain and covariance equations

What is a Jacobian?

For a nonlinear function $f: \mathbb{R}^n \rightarrow \mathbb{R}^n$, the Jacobian $\partial f / \partial x^\top$ is the matrix of all first partial derivatives — entry (i, j) is $\partial f_i / \partial x_j$. It is the best local *linear approximation* to f at a given point: a flat tangent plane that matches f 's slope at that point.

The argument x passed to F_k and H_k should always be the **most recent estimate** of the state: use $\hat{x}_{k|k-1}$ for H_k (before the measurement update) and $\hat{x}_{k|k}$ for F_k (after the measurement update). This keeps the linearisation as accurate as possible.

How linearisation actually works — a 1D example.

Say the state equation is $f(x) = \sin(x)$. That is nonlinear — the KF cannot handle it directly. But zoom in close to any specific point and a curve looks like a straight line. Linearisation is just replacing the curve with the **tangent line** at the current best guess, via a Taylor expansion around $\hat{x}$:

$$f(x) \approx \underbrace{f(\hat{x})}_{(1)} + \underbrace{F_k}_{df/dx|_{\hat{x}}} \underbrace{(x - \hat{x})}_{(2)}$$

For $\sin(x)$ the slope is $\cos(\hat{x})$, so if $\hat{x} = 0.1$:

$$\sin(x) \approx \sin(0.1) + \cos(0.1) \cdot (x - 0.1)$$

That is a straight line — the KF can work with that.

The two parts play different roles in the EKF:

- **Part (1) — mean update.** x is replaced by the estimate: $\hat{x}_{k+1|k} = f(\hat{x}_{k|k}, u_k)$. Plug the best guess directly into the nonlinear function.

- **Part (2) — covariance update.** x here is the *true* (unknown) state. The difference $(x - \hat{x})$ is the error. Propagating this error through the linearised function gives:

$$(x_{k+1} - \hat{x}_{k+1|k}) \approx F_k (x_k - \hat{x}_{k|k}) + w_k$$

Square and take the expectation $\Rightarrow$ the error never appears explicitly; it disappears into the covariance:

$$P_{k+1|k} = F_k P_{k|k} F_k^\top + Q_k$$

Why the most recent estimate? Because the tangent-line approximation is only accurate *near* the expansion point. Using the most recent $\hat{x}_{k|k}$ puts the linearisation exactly where we think the true state is. A fixed expansion point chosen at the start would drift further and further from the truth as the state evolves.

F_k and H_k are **not constant** — they are recomputed at every step by plugging the current state estimate into the derivative formula.

EKF algorithm for the nonlinear DD problem

The EKF has the same two-step structure as the linear KF. To see exactly what changed, each equation below is shown side by side with its linear KF counterpart.

Initial conditions:

$$x_0 \sim \mathcal{N}(\hat{x}_{0|-1}, P_{0|-1})$$

Step B — Measurement update (after receiving y_k and u_k):

	Linear KF		EKF
$\hat{y}_{k k-1}$	$H_k \hat{x}_{k k-1} + D_k u_k$		$h(\hat{x}_{k k-1}, u_k)$
$\tilde{y}_{k k-1}$	$y_k - \hat{y}_{k k-1}$		$y_k - \hat{y}_{k k-1}$
K_k	$P_{k k-1} H_k^\top (H_k P_{k k-1} H_k^\top + R_k)^{-1}$	+	$P_{k k-1} H_k^\top (H_k P_{k k-1} H_k^\top + R_k)^{-1}$ +
$\hat{x}_{k k}$	$\hat{x}_{k k-1} + K_k \tilde{y}_{k k-1}$		$\hat{x}_{k k-1} + K_k \tilde{y}_{k k-1}$
$P_{k k}$	$(I - K_k H_k) P_{k k-1} (I - K_k H_k)^\top + K_k R_k K_k^\top$		$(I - K_k H_k) P_{k k-1} (I - K_k H_k)^\top + K_k R_k K_k^\top$

What changed in Step B?

Only the **first line** — the predicted measurement $\hat{y}_{k|k-1}$.

- Linear KF: $H_k \hat{x}_{k|k-1}$ — a fixed matrix multiplied by the state estimate. Simple and fast.
- EKF: $h(\hat{x}_{k|k-1}, u_k)$ — the actual nonlinear sensor function evaluated at the current estimate. This is the **mean update using h directly**.

Everything else (K_k , $\hat{x}_{k|k}$, $P_{k|k}$) looks identical — *except* that the H_k appearing in those formulas is now the **Jacobian** $H_k = \partial h / \partial x^\top|_{\hat{x}_{k|k-1}}$, not a fixed measurement

matrix. It is recomputed every step from the current state estimate.

Step A — Time update (propagate from k to $k + 1$):

	Linear KF	EKF
$\hat{x}_{k+1 k}$	$\Phi_k \hat{x}_{k k} + \Gamma_k u_k$	$f(\hat{x}_{k k}, u_k)$
$P_{k+1 k}$	$\Phi_k P_{k k} \Phi_k^\top + Q_k$	$F_k P_{k k} F_k^\top + Q_k$

What changed in Step A?

Both lines changed:

- **State prediction** (mean update): $\Phi_k \hat{x}_{k|k} + \Gamma_k u_k \longrightarrow f(\hat{x}_{k|k}, u_k)$.
The fixed matrix Φ_k is gone. Instead, the current estimate is plugged *directly* into the nonlinear state function f — this is what “ f appears directly in the mean update” means.
- **Covariance prediction**: $\Phi_k P_{k|k} \Phi_k^\top + Q_k \longrightarrow F_k P_{k|k} F_k^\top + Q_k$.
The fixed matrix Φ_k is replaced by the Jacobian $F_k = \partial f / \partial x^\top|_{\hat{x}_{k|k}}$, recomputed every step. The structure of the equation is otherwise identical.

Block diagram interpretation

The EKF splits into two parallel loops — one tracking the **mean** (state estimate) and one tracking the **covariance** (uncertainty). They share information: the mean part sends its current estimates to the covariance part so the Jacobians can be evaluated there, and the covariance part sends K_k back to the mean part so the correction can be applied.

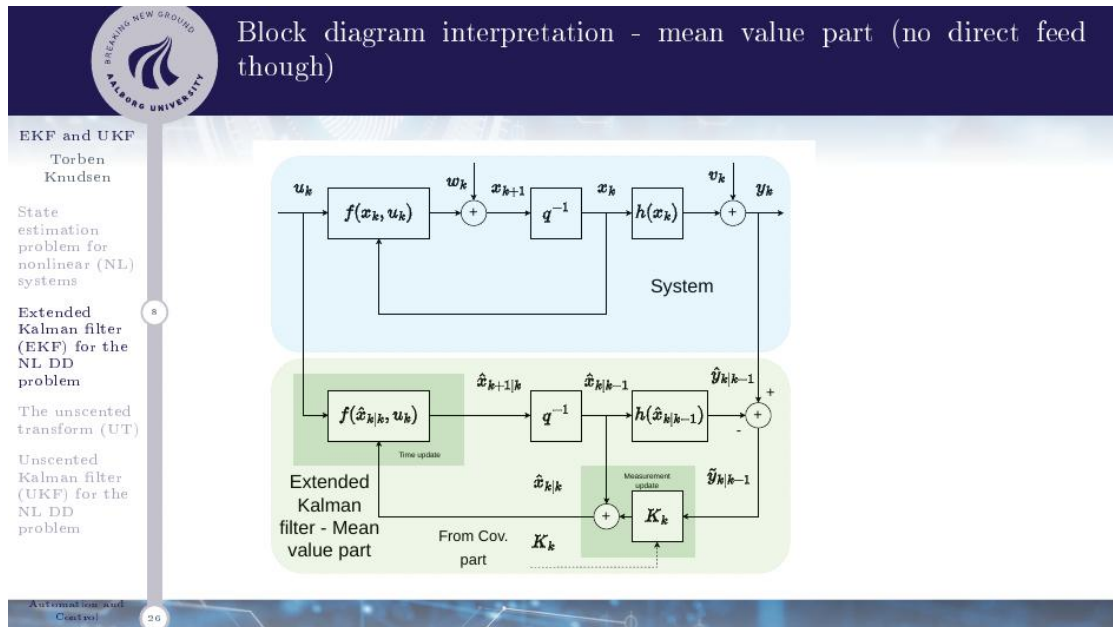


Figure 1: EKF — mean value part.

Reading the mean value diagram (top = system, bottom = EKF):

- **Top half (blue) — the real system.** u_k feeds into $f(x_k, u_k)$, process noise w_k is added, then q^{-1} (a one-step delay) gives the next true state x_k . That state goes into $h(x_k)$, sensor noise v_k is added, and the sensor outputs the real measurement y_k .
- **Bottom half (green) — the EKF mean part.** This mirrors the system but uses estimates throughout.
 - **Time update (left):** $f(\hat{x}_{k|k}, u_k)$ propagates the estimate forward through the actual nonlinear f . q^{-1} gives $\hat{x}_{k+1|k}$ (predicted next state).
 - **Measurement update (right):** $h(\hat{x}_{k|k-1})$ predicts what the sensor *should* read. The real y_k minus this prediction gives the innovation $\tilde{y}_{k|k-1}$. Multiplied by K_k (arriving from the covariance part) and added back to $\hat{x}_{k|k-1}$, this corrects the estimate to $\hat{x}_{k|k}$.

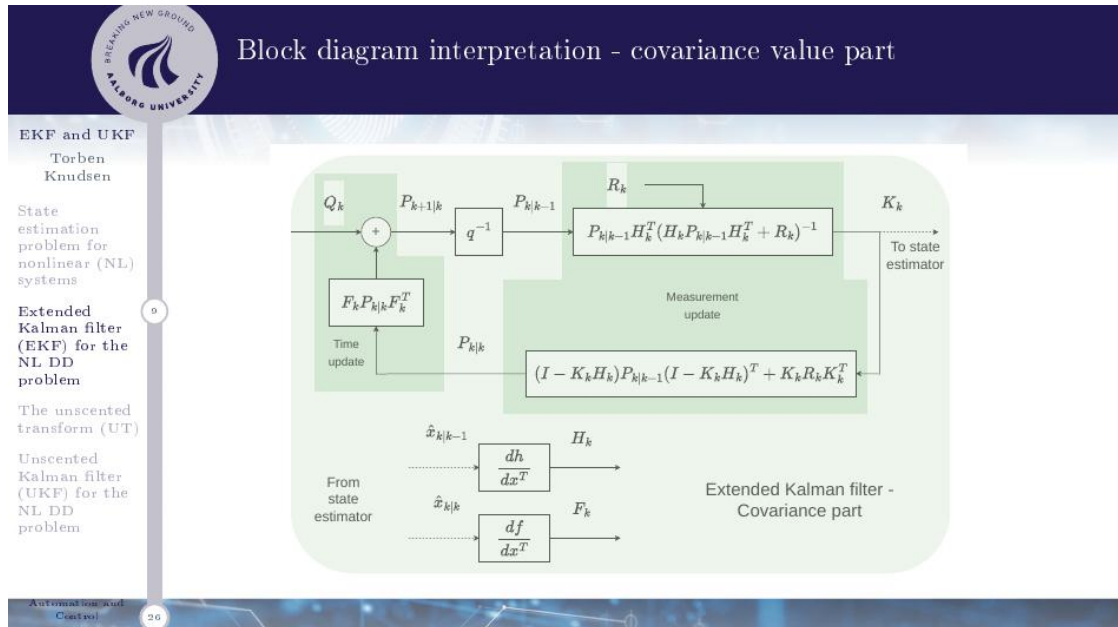


Figure 2: EKF — covariance part.

Reading the covariance diagram:

- **Time update (left loop):** $P_{k|k}$ is multiplied by the Jacobian F_k on both sides and Q_k is added, giving $P_{k+1|k} = F_k P_{k|k} F_k^T + Q_k$. The q^{-1} block feeds this forward to the next step.
- **Measurement update (right loop):** $P_{k|k-1}$ passes through the measurement update formula $(I - K_k H_k) P_{k|k-1} (I - K_k H_k)^T + K_k R_k K_k^T$, giving the updated $P_{k|k}$. Along the way the gain $K_k = P_{k|k-1} H_k^T (H_k P_{k|k-1} H_k^T + R_k)^{-1}$ is computed and sent to the mean-value part.
- **Jacobians from the state estimator (bottom):** The current state estimates

$\hat{x}_{k|k-1}$ and $\hat{x}_{k|k}$ are fed in from the mean-value part. $H_k = \partial h / \partial x$ is evaluated at $\hat{x}_{k|k-1}$ and $F_k = \partial f / \partial x$ at $\hat{x}_{k|k}$ — recomputed every step so the linearisation stays accurate.

Important difficulties with the EKF

The EKF has two fundamental limitations:

- **Not optimal.** The EKF is only an approximation — it linearises the nonlinear functions and then applies the (optimal) KF to that approximation. The linearisation introduces errors, so the EKF result is in general *not* the true conditional mean $E[x_k | \mathcal{Y}_k]$.
- **Not guaranteed to be stable.** In the linear KF, stability can be checked analytically (observability, controllability conditions). For the EKF there is no general stability guarantee — if the linearisation is poor (highly nonlinear region, bad initial estimate), the filter can diverge.

Both limitations motivate the **UKF** (Section 3), which avoids linearisation entirely by propagating a set of carefully chosen sample points through the actual nonlinear function.

Key takeaways — Section 2

- **Beyond linear combinations.** The linear KF is restricted to matrices — state and input can only enter the equations as $\Phi_k x_k + \Gamma_k u_k$ (a linear combination). The EKF removes this restriction: $f(x_k, u_k)$ and $h(x_k, u_k)$ can be *any* function of the state and input — sin, $\sqrt{\cdot}$, products, anything the physics requires.
- **Jacobians provide the local linearisation.** Since the KF machinery needs linear equations, we linearise f and h at the current estimate using partial derivatives: $F_k = \partial f / \partial x^\top |_{\hat{x}_{k|k}}$ and $H_k = \partial h / \partial x^\top |_{\hat{x}_{k|k-1}}$. Each Jacobian gives the tangent-plane approximation of its function at that point — recomputed every step as the estimate moves.
- **Nonlinear functions used directly for the mean.** The state prediction $\hat{x}_{k+1|k} = f(\hat{x}_{k|k}, u_k)$ and measurement prediction $\hat{y}_{k|k-1} = h(\hat{x}_{k|k-1}, u_k)$ use the actual nonlinear functions — not approximations. This keeps the estimated trajectory as close to the true physics as possible.
- **Jacobians replace matrices in the covariance and gain.** Wherever the linear KF used the fixed matrices Φ_k and H_k , the EKF substitutes the Jacobians F_k and H_k . This includes the covariance time update, the covariance measurement update, and the Kalman gain K_k — since H_k appears inside the gain formula as well.
- **Not optimal and not guaranteed stable.** The Jacobian is only a local approximation — wherever the true function curves away from the tangent, an error is introduced. This means the EKF does not achieve the true conditional mean, so it is not optimal. It also means stability cannot be verified analytically the way it can for the linear KF — if the linearisation is poor (highly nonlinear system or a bad initial estimate), the filter can diverge.

Section 2b What the EKF Is Really Doing

The filter tracks two things: mean and covariance

The Kalman filter never knows the true state x_k exactly. Instead it maintains a Gaussian probability distribution over where the state might be — a bell curve of belief. A Gaussian is fully described by just two parameters:

- **Mean** $\hat{x}_k$ — the centre of the distribution, i.e. the best guess of the state.
- **Covariance** P_k — the spread of the distribution, i.e. how uncertain that guess is. Formally, $P_k = E[(x_k - \hat{x}_k)(x_k - \hat{x}_k)^\top]$ — the expected squared error of the estimate.

Both must be updated at every step. That is why each step of the filter has two equations, one for the mean and one for the covariance. **Important:** P_k says nothing about how variable the physical state itself is — it purely measures how well we *know* the state. A pendulum swinging wildly can still have a small P_k if the sensors are good.

Time update: predict forward

- **Mean** — push the best guess forward through the nonlinear model: $\hat{x}_{k+1|k} = f(\hat{x}_{k|k}, u_k)$. The mean is a single point, so you simply evaluate f at it. This is the physics doing its job: given where we think the state is now, where does the model say it will be next step?
- **Covariance** — uncertainty *grows*: $P_{k+1|k} = F_k P_{k|k} F_k^\top + Q_k$. Two reasons: (1) the model f is never perfect — unmodelled forces, friction, flex — so prediction error accumulates. (2) process noise Q_k represents random disturbances that no model can predict. Q_k is tuned large when you distrust the model, small when you trust it.

Measurement update: correct with sensor data

- **Mean** — the sensor gives real evidence about the true state. Compare what the sensor actually read (y_k) to what we predicted it would read ($h(\hat{x}_{k|k-1}, u_k)$). The difference is the innovation $\tilde{y}_{k|k-1}$, and the mean is corrected by $K_k \tilde{y}_{k|k-1}$: a nudge in the direction the measurement says we were wrong.
- **Covariance** — uncertainty *shrinks*. A measurement is new information about where the state is, so we become more certain after it arrives. How much it shrinks depends on K_k : a noisy sensor (large R_k) gives a small K_k so the covariance barely moves; a precise sensor gives a large K_k and a bigger drop in uncertainty.

The rhythm of the filter. Every step is the same battle: time update makes uncertainty grow (we predicted, and predictions are imperfect), then the measurement update makes it shrink (we measured, and measurements contain information). The covariance settles at a steady value where these two forces balance. The mean meanwhile keeps chasing the true state: time update moves it with the physics, measurement update corrects any drift.

Why the mean can use f directly but covariance cannot

This is the central insight of the EKF.

The mean is a single point $\hat{x}_{k|k}$. You just evaluate f at that point and get the predicted mean. Done. It is a reasonable approximation because the mean is the most likely location of the state, so running it through f gives the most likely next location.

The covariance is not a point — it describes the *shape and spread* of the entire Gaussian cloud around the mean. To propagate it you need to know how f stretches and rotates that cloud. For a linear function $y = Ax$ this is exact: $\text{Cov}(y) = APA^\top$. For a nonlinear f there is no such formula — passing a Gaussian through a nonlinear function produces a distribution that is no longer Gaussian; it becomes skewed and asymmetric because f compresses space on one side and stretches it on the other.

The Jacobian $F_k = \partial f / \partial x^\top|_{\hat{x}_{k|k}}$ fixes this by giving the best local linear approximation to how f stretches space *at the mean point*. Now the same $APA^\top$ structure applies: $P_{k+1|k} \approx F_k P_{k|k} F_k^\top + Q_k$. This is the same Gaussian-to-Gaussian formula, justified by pretending f is locally flat at $\hat{x}$.

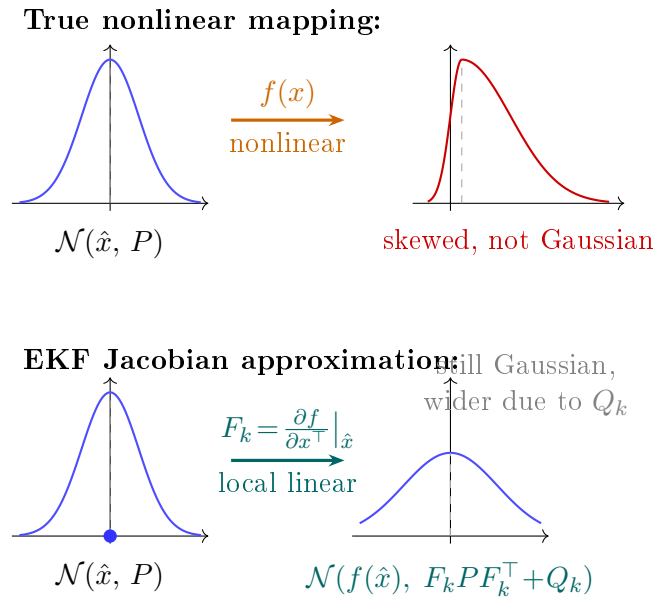


Figure 3: Top: passing a Gaussian through a nonlinear f produces a skewed, non-Gaussian output — no simple covariance formula exists. Bottom: the EKF approximates f as locally linear at $\hat{x}$ using the Jacobian F_k , so the output remains Gaussian and the covariance can be propagated with $F_k P F_k^\top + Q_k$. The mean is simply evaluated: $f(\hat{x})$ gives the new centre directly.

The EKF in one paragraph. The filter tracks a Gaussian belief $\mathcal{N}(\hat{x}_k, P_k)$ about the state. At each step, the **time update** predicts forward: the mean moves through the nonlinear f directly ($\hat{x}_{k+1|k} = f(\hat{x}_{k|k}, u_k)$) while the covariance grows through the Jacobian F_k plus process noise ($P_{k+1|k} = F_k P_{k|k} F_k^\top + Q_k$) — uncertainty increases because the model is imperfect and the world is noisy. The **measurement update** then corrects both: the mean is nudged towards what the sensor said, and the covari-

ance shrinks because a measurement is new information. The Jacobian F_k is needed for the covariance because a Gaussian pushed through a nonlinear curve is no longer Gaussian — the Jacobian linearises f locally so the Gaussian-to-Gaussian propagation formula still applies. The mean can use f directly because a point just needs to be evaluated, not transformed as a distribution.

Section 3 The Unscented Transform (UT)

Motivation: what problem does the UT solve?

Both the EKF and UKF need to answer the same core question at every step:

Given a random variable x with known mean μ_x and covariance C_x , what are the mean and covariance of $y = f(x)$ after passing through a nonlinear function f ?

For a **linear** f this is easy — matrices pass means and covariances through exactly. For a **nonlinear** f there is no closed-form answer in general. The EKF solves it by approximating f as linear (Jacobian). The UT takes a completely different approach: keep f *exact* and instead choose a small, carefully placed set of sample points — called **sigma points** — that capture the mean and covariance of x , push each one through the actual nonlinear f , and estimate the output statistics from the results.

Three approaches compared

Approximate first and second moments for a stochastic function

EKF and UKF
Torben Knudsen
State estimation problem for nonlinear (NL) systems
Extended Kalman filter (EKF) for the NL DD problem

The unscented transform (UT)
Unscented Kalman filter (UKF) for the NL DD problem

The generic probabilistic problem
Calculate/approximate the second order statistics of $x, y = f(x)$ given the second order statistics of x and the function f :

$$y = f(x), \mu_x = E(x), C_x = \text{Cov}(x), x \in R^n, y \in R^p$$

For non-linear relations there is no exact solution in general.

The linearization solution
The EKF can be interpreted as using the linearization approach:

$$y = f(\mu_x) + \nabla f(\mu_x)(x - \mu_x) \Rightarrow$$

$$\mu_y = f(\mu_x), C_{yx} = \nabla f(\mu_x)C_x,$$

$$C_y = \nabla f(\mu_x)C_x \nabla f(\mu_x)^T.$$

Automation and Control 26

Figure 4: The generic probabilistic problem and the linearisation (EKF) solution.

Method	How it works	Key point
Linearisation (EKF)	Replace f with its tangent (Jacobian) at μ_x .	Fast, but introduces error wherever f curves.
Monte Carlo (MC)	Draw thousands of random samples x_i , compute $f(x_i)$ for each, estimate mean and covariance from the sample.	Accurate but needs very many samples — expensive.
Unscented Transform	Choose $2n + 1$ deterministic sigma points that exactly represent μ_x and C_x , push each through f , form weighted mean and covariance of the outputs.	No Jacobians, uses actual f , needs only $2n + 1$ points.

The UT is essentially Monte Carlo, but the samples are **not random** — they are constructed deterministically to encode exactly the right mean and covariance. Because they are chosen optimally, only $2n + 1$ points are needed (where n is the state dimension) instead of the thousands required by MC.

UT algorithm: sigma points and weights



EKF and UKF
Torben Knudsen

State estimation problem for nonlinear (NL) systems

Extended Kalman filter (EKF) for the NL DD problem

The unscented transform (UT)

Unscented Kalman filter (UKF) for the NL DD problem

UT Algorithm

$\alpha = 1, \kappa = 2, \beta = 0$, (Default parameters gives $\lambda = 2$)

$\lambda = \alpha^2(n + \kappa) - n, k = \sqrt{n + \lambda}, n = \dim(x)$,

$u_i \triangleq$ eigenvector i for $C_x, l_i \triangleq$ eigenvalue i for C_x ,

$$x_i = \begin{cases} \mu_x & , i = 0 \\ \mu_x + k u_i \sqrt{l_i} & , 1 \leq i \leq n \\ \mu_x - k u_i \sqrt{l_i} & , n + 1 \leq i \leq 2n \end{cases}$$

$$w_i^t = \begin{cases} \frac{\lambda}{n + \lambda} & , i = 0, t = m \text{ (for mean)} \\ \frac{\lambda}{n + \lambda} + 1 - \alpha^2 + \beta & , i = 0, t = c \text{ (for covarians)} \\ \frac{1}{2(n + \lambda)} & , 1 \leq i \leq 2n \end{cases}$$

$$\hat{\mu}_y = \sum_{i=0}^{2n} w_i^m f(x_i), \hat{C}_{yx} = \sum_{i=0}^{2n} w_i^c (f(x_i) - \hat{\mu}_y)(x_i - \hat{\mu}_x)^T,$$

$$\hat{C}_y = \sum_{i=0}^{2n} w_i^c (f(x_i) - \hat{\mu}_y)(f(x_i) - \hat{\mu}_y)^T.$$

Figure 5: The full UT algorithm.

Step 1 — Parameters (default values $\alpha = 1$, $\kappa = 2$, $\beta = 0$ give $\lambda = 2$):

$$\lambda = \alpha^2(n + \kappa) - n, \quad k = \sqrt{n + \lambda}, \quad n = \dim(x)$$

λ is a scaling parameter controlling how far the sigma points are spread from the mean. k is the actual distance multiplier used when placing them. With the default values and $n = 2$ states: $\lambda = 1^2(2 + 2) - 2 = 2$, $k = \sqrt{2 + 2} = 2$. The larger n is, the further out the points are placed to cover the wider spread of a higher-dimensional Gaussian.

Step 2 — Sigma points (eigenvalue decomposition version). The covariance matrix C_x is decomposed into its eigenvectors u_i and eigenvalues l_i — this is called the **eigenvalue decomposition** (also called spectral decomposition). Let u_i be the i -th eigenvector and l_i the i -th eigenvalue of C_x . The $2n + 1$ sigma points are:

$$x_i = \begin{cases} \mu_x & i = 0 \\ \mu_x + k u_i \sqrt{l_i} & 1 \leq i \leq n \\ \mu_x - k u_i \sqrt{l_i} & n + 1 \leq i \leq 2n \end{cases}$$

What are the sigma points — concretely?

Say you have a 2D state ($n = 2$), so $2n + 1 = 5$ sigma points total:

- $x_0 = \mu_x$ — the mean itself, one point at the centre
- $x_1 = \mu_x + k u_1 \sqrt{l_1}$ — pushed out along the first principal axis of the covariance ellipse
- $x_2 = \mu_x + k u_2 \sqrt{l_2}$ — pushed out along the second principal axis
- $x_3 = \mu_x - k u_1 \sqrt{l_1}$ — mirror of x_1
- $x_4 = \mu_x - k u_2 \sqrt{l_2}$ — mirror of x_2

The eigenvectors u_i point along the axes of the covariance ellipse (the directions of greatest and least uncertainty). The eigenvalues l_i give the variance in each direction — so $\sqrt{l_i}$ is roughly the standard deviation. Multiplying by k controls how many standard deviations out the points sit. Together these five points encode exactly the same mean and covariance as the original Gaussian — no random sampling needed.

Step 3 — Weights. Each sigma point gets a weight for computing the output mean (w^m) and a separate weight for computing the output covariance (w^c). The centre point x_0 gets a different weight from the others because it sits at the mean and carries more representative information:

$$w_i^m = \begin{cases} \frac{\lambda}{n + \lambda} & i = 0 \\ \frac{1}{2(n + \lambda)} & 1 \leq i \leq 2n \end{cases} \quad w_i^c = \begin{cases} \frac{\lambda}{n + \lambda} + 1 - \alpha^2 + \beta & i = 0 \\ \frac{1}{2(n + \lambda)} & 1 \leq i \leq 2n \end{cases}$$

Weights: what they are and when they change.

Are the weights fixed? Yes — once you choose α , κ , β and know n (the state dimension), all weights are computed once and stay constant for every step of the filter. They do not depend on the current state estimate or covariance.

Why two sets? w_i^m controls how much each mapped sigma point contributes to the *output mean* estimate. w_i^c controls how much each contributes to the *output covariance* estimate. For the outer points ($i \geq 1$) the two weights are identical. Only the centre point $i = 0$ differs, because the extra term $1 - \alpha^2 + \beta$ allows tuning how much the centre point influences the covariance independently of the mean.

Numerical example with default values ($n = 2$, $\alpha = 1$, $\kappa = 2$, $\beta = 0$, so $\lambda = 2$):

- *Mean weights:* $w_0^m = \frac{\lambda}{n + \lambda} = \frac{2}{4} = 0.5$, $w_{1...4}^m = \frac{1}{2(n + \lambda)} = \frac{1}{8} = 0.125$
Check: $0.5 + 4 \times 0.125 = 1$ ✓
- *Covariance weights:* $w_0^c = \frac{\lambda}{n + \lambda} + 1 - \alpha^2 + \beta = 0.5 + 1 - 1 + 0 = 0.5$, $w_{1...4}^c = 0.125$
Check: $0.5 + 4 \times 0.125 = 1$ ✓

With default $\beta = 0$ and $\alpha = 1$, the extra term is zero so $w_0^c = w_0^m$ — the centre point has the same influence on both mean and covariance. Setting $\beta = 2$ (optimal for a Gaussian input) gives $w_0^c = 0.5 + 1 - 1 + 2 = 2.5$ — the centre point gets much more weight for covariance, which improves accuracy when the input really is Gaussian. The outer weights stay at 0.125 regardless.

Step 4 — Propagate and compute output statistics. Push each sigma point through the actual nonlinear f (no Jacobian, no approximation), then form weighted sums:

Output mean — weighted average of where the sigma points land after f :

$$\hat{\mu}_y = \sum_{i=0}^{2n} w_i^m f(x_i)$$

Output cross-covariance — weighted measure of how much the output $f(x_i)$ and the input x_i move together (needed for the Kalman gain in the UKF):

$$\hat{C}_{yx} = \sum_{i=0}^{2n} w_i^c (f(x_i) - \hat{\mu}_y)(x_i - \mu_x)^\top$$

Output covariance — weighted spread of the mapped sigma points around the estimated output mean:

$$\hat{C}_y = \sum_{i=0}^{2n} w_i^c (f(x_i) - \hat{\mu}_y)(f(x_i) - \hat{\mu}_y)^\top$$

What do these three output quantities give you?

- $\hat{\mu}_y$ — estimated mean of $y = f(x)$ (weighted average of where the sigma points land after f).

- $\hat{C}_y$ — estimated covariance of y (weighted spread of the mapped sigma points around $\hat{\mu}_y$).
- $\hat{C}_{yx}$ — estimated cross-covariance between y and x (how much they move together) — this is needed for the Kalman gain in the UKF.

Important properties of the UT

- **Exact for affine functions.** If $f(x) = Ax + b$ (linear plus offset), the UT gives the exact mean and covariance — same as the standard KF would. This is a consistency check: the UT should at least match the KF when the system is actually linear.
- **Good approximation for nonlinear f .** The UT captures the mean and covariance correctly to second order in a Taylor expansion, whereas the EKF (linearisation) is only accurate to first order. This means the UT is generally **more accurate** than the EKF, especially for highly nonlinear systems.
- **No Jacobians needed.** f is used as a black box — you never compute $\partial f / \partial x$. This is a major practical advantage when f is complicated or the derivative is hard to derive.
- **Generally superior to both linearisation and MC.** Better accuracy than EKF with far fewer evaluations of f than MC ($2n + 1$ vs thousands).

Example and comparison plots

The test case uses $y = x^\top x$ with $x \in \mathbb{R}^2$ — a 2D state with $E(x) = [1, 1]^\top$ and $\text{Cov}(x) = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$. The function $y = x^\top x = x_1^2 + x_2^2$ is a scalar output — the squared distance from the origin. It is nonlinear (involves products of states), so the output distribution is not Gaussian. The correct answer is known analytically, so errors can be measured.

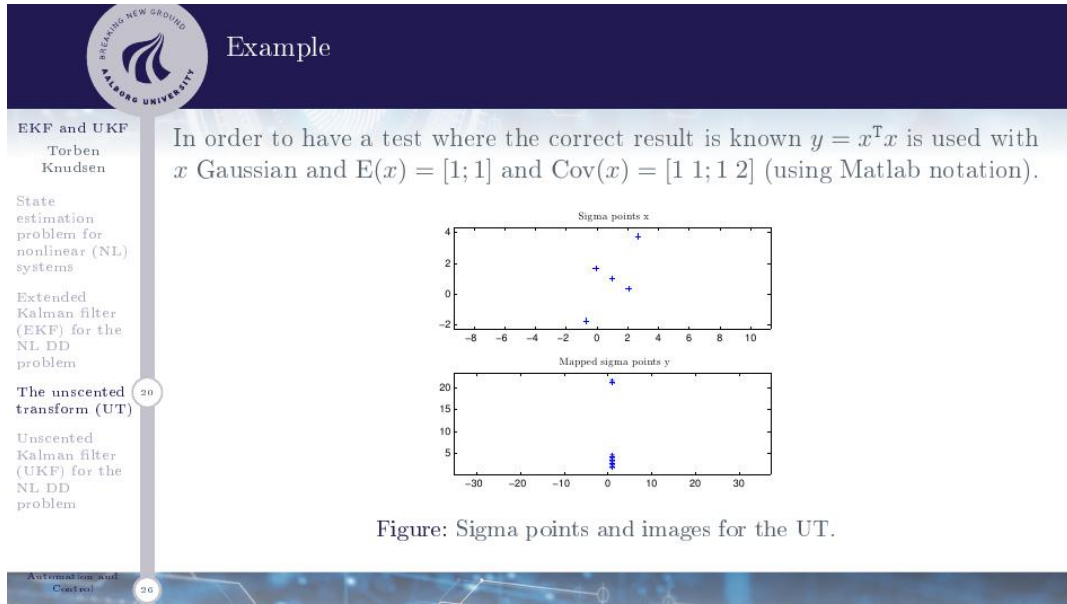


Figure 6: UT: sigma points in input space (top) and their mapped values in output space (bottom). $n = 2$ states so $2n + 1 = 5$ sigma points total. **Top plot** — axes are x_1 and x_2 (the two state components). The 5 sigma points are shown as + marks: one at the mean $[1, 1]^T$ and four pushed out symmetrically along the principal axes of the covariance ellipse. **Bottom plot** — each sigma point has been put through f , giving a scalar $y_i = x_{i,1}^2 + x_{i,2}^2$. The 5 output values are shown on the y -axis as + marks. The weighted mean and variance of these 5 numbers gives the UT estimate of the output distribution.

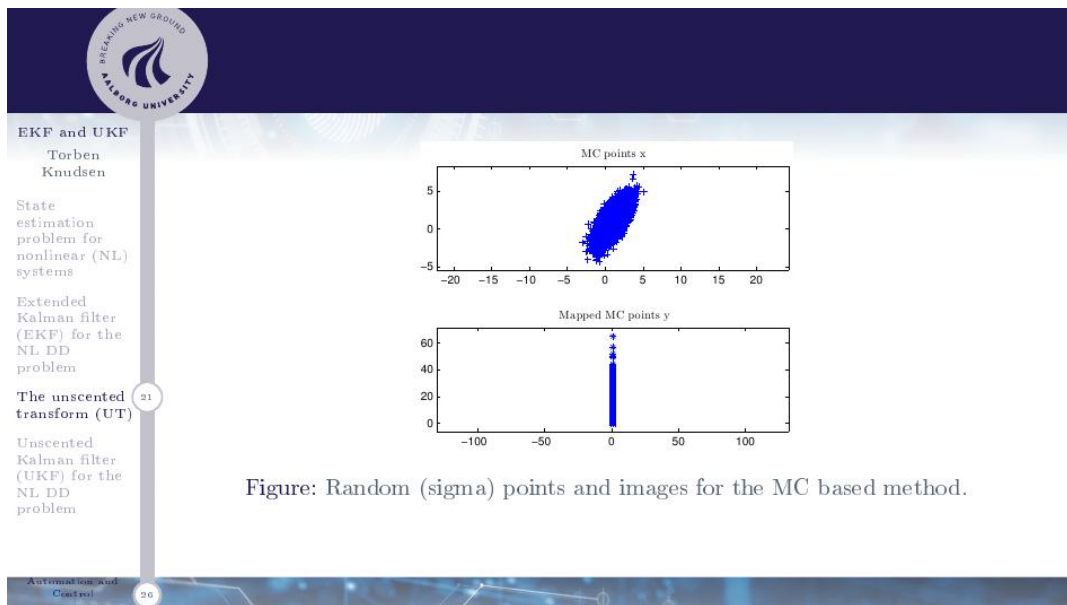


Figure 7: MC comparison using thousands of random samples. **Top plot** — axes are again x_1 and x_2 . Instead of 5 chosen points, thousands of random samples are drawn from $\mathcal{N}([1, 1]^T, C_x)$ — the dense blue cloud. **Bottom plot** — every sample is put through f , giving a cloud of scalar y values along the axis. The sample mean and variance of this cloud estimates the output statistics. The UT (5 points) achieves essentially the same answer as MC (thousands of points).

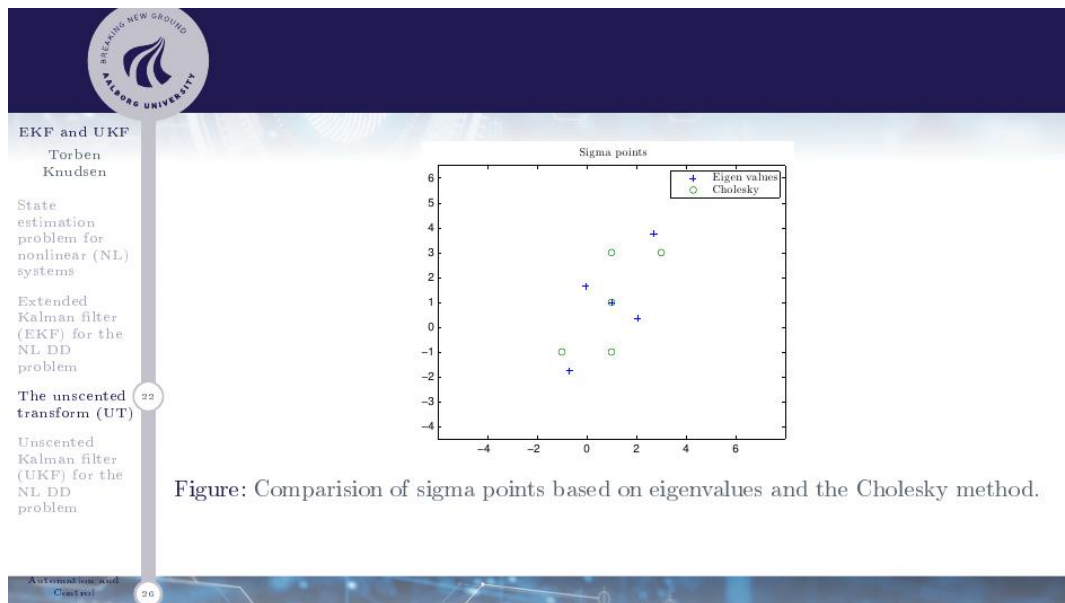


Figure 8: Comparison of sigma point placement: **eigenvalue decomposition** (blue +, the version used in the algorithm above) vs. **Cholesky decomposition** (green o, an alternative). Both place points along the principal directions of the covariance ellipse, but they arrive at those directions differently. The eigenvalue method decomposes C_x into eigenvectors and eigenvalues directly. The Cholesky method instead factors $C_x = LL^T$ (where L is a lower-triangular matrix) and uses the columns of L to place the points — no eigenvectors needed, which is faster to compute. Both give valid sigma points and the same final estimates; Cholesky is the more common choice in practice for this reason.

Method	My	Cy	Cyx	
UTSCEst	5.000e+0	3.900e+1	4.000e+0	6.000e+0
UTJMEst	5.000e+0	3.100e+1	4.000e+0	6.000e+0
MCEst	5.038e+0	3.385e+1	3.988e+0	6.028e+0
THTK	5.000e+0	3.400e+1	4.000e+0	6.000e+0
UTSCRelE	0	1.471e-1	0	0
UTJMRelE	0	-8.824e-2	0	0
MCRRelE	7.663e-3	-4.284e-3	-3.090e-3	4.670e-3

Table: Performance for the above method, the Cholesky based, the MC and the theoretical. The rows marked RelE are the errors relative to the correct results.

Figure 9: Performance comparison table for the test case $y = x^T x$, $x \in \mathbb{R}^2$.

Reading the performance table.

Columns — the four output quantities being estimated:

- M_y — the estimated **mean** of the output y . Correct answer: $E[y] = E[x_1^2 + x_2^2] = 5$.
- C_y — the estimated **variance** of y (since y is a scalar, this is a single number). Correct answer: 34.
- C_{yx} (two numbers) — the estimated **cross-covariance** between y and each component of x : how much does y move when x_1 moves, and how much when x_2 moves. Correct answer: [4, 6].

Rows — the four methods being compared:

- **UTSCEst** — UT using Cholesky decomposition (the green $\circ$ points from the previous figure).
- **UTJMEst** — UT using eigenvalue decomposition (the blue $+$ points; the version in the algorithm above).
- **MCEst** — Monte Carlo estimate using thousands of random samples.
- **THTK** — the theoretical correct answer, computed analytically. This is the target every method is trying to hit.

RelE rows — relative error of each method compared to THTK, i.e. (estimate – correct)/correct. Zero means perfect.

What to take away:

- Both UT methods get M_y and C_{yx} **exactly right** (RelE = 0) — the mean and cross-covariance are perfect.
- There is a small error in C_y (the variance) — around 15% for UTSC and –9% for UTJM. This is expected: the UT is not exact for nonlinear f , but it is a good approximation.
- MC has small errors *everywhere* ($< 1\%$) because it uses very many samples, but it achieves this at the cost of thousands of function evaluations versus just 5 for the UT.

Section 4 The Unscented Kalman Filter (UKF)

From UT to UKF: the key idea

The UT solves the core problem of propagating a mean and covariance through a nonlinear function. The UKF simply **plugs the UT into the general KF equations** wherever a nonlinear propagation is needed — once for the measurement function h (Step B) and once for the state function f (Step A). No Jacobians anywhere.

To keep the notation compact, the UT is written as a single function U :

$$\begin{bmatrix} E(y) \\ \text{Cov}(y, x) \\ \text{Cov}(y) \end{bmatrix} = U(f, E(x), \text{Cov}(x))$$

In words: give U the function, the current mean, and the current covariance — it runs the sigma-point machinery internally and returns the output mean, the output covariance, and the cross-covariance.

The same formula works when everything is **conditioned on some known information** z :

$$\begin{bmatrix} E(y | z) \\ \text{Cov}(y, x | z) \\ \text{Cov}(y | z) \end{bmatrix} = U(f, E(x | z), \text{Cov}(x | z))$$

What does “conditioned on z ” mean?

Conditioning on z just means: *given that we already know z* , what are the statistics? If $z = \mathcal{Y}_{k-1}$ (all measurements up to step $k - 1$), then $E(x | z) = \hat{x}_{k|k-1}$ is our estimated mean *taking all past measurements into account*, and $\text{Cov}(x | z) = P_{k|k-1}$ is our uncertainty given that same information.

The point of writing it this way is to show that the UT works identically whether the inputs are raw (no prior information) or conditional on past data — you just feed in whatever mean and covariance you currently have, and the UT gives back the corresponding output statistics. In the KF, z is always the set of past measurements $\mathcal{Y}_{k-1}$, so the outputs are automatically conditioned on everything we have seen so far.

Unscented Kalman filter Definition

Assume the NL map f with input x and output y , the UT can then be represented by the function U below:

$$\begin{bmatrix} E(y) \\ \text{Cov}(y, x) \\ \text{Cov}(y) \end{bmatrix} = U(f, E(x), \text{Cov}(x))$$

If the mean and covariance input is conditioned on say z then the output is also conditioned on z i.e.:

$$\begin{bmatrix} E(y|z) \\ \text{Cov}(y, x|z) \\ \text{Cov}(y|z) \end{bmatrix} = U(f, E(x|z), \text{Cov}(x|z))$$

Figure 10: UKF definition: the UT written as a compact function U .

UKF algorithm

The structure mirrors the EKF exactly — same two-step predict and correct — but every place the EKF used a Jacobian, the UKF calls U instead.

Initial conditions:

$$x_0 \sim \mathcal{N}(\hat{x}_{0|-1}, P_{0|-1})$$

Step B — Measurement update (after receiving y_k and u_k):

First, run the UT on the measurement function h to get the predicted measurement, its covariance, and the cross-covariance with the state:

$$\begin{bmatrix} \hat{y}_{k|k-1} \\ \text{Cov}(y_k, x_k | \mathcal{Y}_{k-1}) \\ C_k^h \end{bmatrix} = U(h, \hat{x}_{k|k-1}, P_{k|k-1})$$

(1) Total measurement uncertainty — the UT gave us C_k^h , the covariance of the predicted measurement due to state uncertainty. But the sensor itself also has noise R_k . Add them together to get the full uncertainty in what the sensor reads:

$$\text{Cov}(y_k | \mathcal{Y}_{k-1}) = C_k^h + R_k$$

(2) Innovation — compare what the sensor actually read (y_k) to what the UT predicted it would read ($\hat{y}_{k|k-1}$). The difference is how wrong the prediction was:

$$\tilde{y}_{k|k-1} = y_k - \hat{y}_{k|k-1}$$

(3) Kalman gain — how much should we trust the innovation? The cross-covariance $\text{Cov}(x_k, y_k)$ (from the UT) tells us how strongly the state and measurement are linked. Dividing by the total measurement uncertainty scales the correction: large sensor noise $\Rightarrow$ small $K_k \Rightarrow$ small correction; precise sensor $\Rightarrow$ large $K_k \Rightarrow$ big correction:

$$K_k = \text{Cov}(x_k, y_k | \mathcal{Y}_{k-1}) \text{Cov}(y_k | \mathcal{Y}_{k-1})^{-1}$$

(4) State mean update — nudge the predicted state estimate in the direction the innovation says we were wrong, scaled by K_k :

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k \tilde{y}_{k|k-1}$$

(5) Covariance update — the measurement gave us new information, so uncertainty shrinks. How much it shrinks depends on how strongly the state and measurement are linked (the cross-covariance from the UT):

$$P_{k|k} = P_{k|k-1} - K_k \text{Cov}(y_k, x_k | \mathcal{Y}_{k-1})$$

UKF Algorithm

EKF and UKF
Torben Knudsen

State estimation problem for nonlinear (NL) systems

Extended Kalman filter (EKF) for the NL DD problem

The unscented transform (UT)

Unscented Kalman filter (UKF) for the NL DD problem

Initial conditions:

$$x_0 \in N(\hat{x}_{0|-1}, P_{0|-1})$$

Measurements update after receiving y_k and u_k :

$$\begin{bmatrix} \hat{y}_{k|k-1} \\ \text{Cov}(y_k, x_k | \mathcal{Y}_{k-1}) \\ C_k^h \end{bmatrix} = U(h, \hat{x}_{k|k-1}, P_{k|k-1})$$

$$\text{Cov}(y_k | \mathcal{Y}_{k-1}) = C_k^h + R_k$$

$$\text{Cov}(x_k, y_k | \mathcal{Y}_{k-1}) = \text{Cov}(y_k, x_k | \mathcal{Y}_{k-1})^T$$

$$\tilde{y}_{k|k-1} = y_k - \hat{y}_{k|k-1}$$

$$K_k = \text{Cov}(x_k, y_k | \mathcal{Y}_{k-1}) \text{Cov}(y_k | \mathcal{Y}_{k-1})^{-1}$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k \tilde{y}_{k|k-1}$$

$$P_{k|k} = P_{k|k-1} - K_k \text{Cov}(y_k, x_k | \mathcal{Y}_{k-1})$$

Automation and Control 20

Figure 11: UKF algorithm — measurement update.

Step A — Time update (propagate from k to $k + 1$):

(1) **Run the UT on f** — feed the current updated estimate $\hat{x}_{k|k}$ and covariance $P_{k|k}$ into the UT together with the state function f . The UT places sigma points around $\hat{x}_{k|k}$, pushes each one through the actual nonlinear f , and returns three things: the predicted next state mean, the unused cross-covariance X , and C_k^f (the covariance of the predicted next state due to state uncertainty alone — process noise not yet included):

$$\begin{bmatrix} \hat{x}_{k+1|k} \\ X \\ C_k^f \end{bmatrix} = U(f, \hat{x}_{k|k}, P_{k|k})$$

(2) **Add process noise** — C_k^f only accounts for how the current uncertainty in x_k spreads through f . It does not yet include the random disturbances that hit the system between steps ($w_k \sim \text{NID}(0, Q_k)$). Adding Q_k gives the full predicted covariance for the next step:

$$P_{k+1|k} = C_k^f + Q_k$$

The slide is titled "Time update from k to $k+1$:" and displays the following equations:

$$\begin{bmatrix} \hat{x}_{k+1|k} \\ X \\ C_k^f \end{bmatrix} = U(f, \hat{x}_{k|k}, P_{k|k}),$$

$$P_{k+1|k} = \text{Cov}(x_{k+1} | \mathcal{Y}_k) = C_k^f + Q_k.$$

The sidebar on the left contains the following text:

- EKF and UKF
- Torben Knudsen
- State estimation problem for nonlinear (NL) systems
- Extended Kalman filter (EKF) for the NL DD problem
- The unscented transform (UT)
- Unscented Kalman filter (UKF) for the NL DD problem

The footer includes the Aalborg University logo and the text "Automation and Control".

Figure 12: UKF algorithm — time update.

What is X in the time update?

X is the cross-covariance $\text{Cov}(x_{k+1}, x_k | \mathcal{Y}_k)$ — how much the next state and the current state move together. It is an output of U but is **not used** in the standard UKF time update equations; only $\hat{x}_{k+1|k}$ and C_k^f are needed. It would only matter in more advanced formulations.

EKF vs UKF side by side

The $\dagger$ symbol marks quantities that come directly out of the UT call U — no extra computation needed.

	EKF	UKF
Mean (B)	update $\hat{y}_{k k-1} = h(\hat{x}_{k k-1}, u_k)$ single point evaluation of h	$\hat{y}_{k k-1}^\dagger$ from $U(h, \hat{x}_{k k-1}, P_{k k-1})$ weighted mean of sigma points through h
Cov. (B)	update $(I - K_k H_k)P(I - K_k H_k)^\top + K_k R_k K_k^\top$ $H_k = \text{Jacobian of } h$	$P_{k k-1} - K_k \text{Cov}(y_k, x_k)^\dagger$ $\text{Cov}(y_k, x_k)$ from U , no Jacobian
Gain K_k	$P_{k k-1} H_k^\top (H_k P_{k k-1} H_k^\top + R_k)^{-1}$ built from Jacobian H_k	$\text{Cov}(x_k, y_k)^\dagger \text{Cov}(y_k)^{-1}$ both covariances come from U directly
Mean (A)	update $\hat{x}_{k+1 k} = f(\hat{x}_{k k}, u_k)$ single point evaluation of f	$\hat{x}_{k+1 k}^\dagger$ from $U(f, \hat{x}_{k k}, P_{k k})$ weighted mean of sigma points through f
Cov. (A)	update $F_k P_{k k} F_k^\top + Q_k$ $F_k = \text{Jacobian of } f$	$C_k^{f\dagger} + Q_k$ C_k^f from U , Q_k added for process noise
Jacobians?	Yes — F_k and H_k every step	No — f and h are black boxes
Optimal?	No	No
Stable?	Not guaranteed	Not guaranteed

Key takeaways — Section 4

- The UKF replaces **both** the mean update and the covariance update with a single call to U . In the EKF these are two separate jobs: evaluate f or h at one point for the mean, then use the Jacobian for the covariance. In the UKF, one UT call handles both at once — push $2n + 1$ sigma points through the actual f or h , and you immediately get the output mean, output covariance, and cross-covariance all together.
- The structure is identical to the EKF: two steps, same gain formula, same correction logic. Only the *mechanism* for computing means and covariances is different.
- The UKF Kalman gain uses $\text{Cov}(x_k, y_k)$ and $\text{Cov}(y_k)$ directly — these come out of the UT automatically, no matrix inversion of a Jacobian needed.
- Like the EKF, the UKF is **not optimal** and **not guaranteed stable**.

Why the UKF is more accurate than the EKF — Taylor expansion argument.

Why the EKF is only first-order accurate.

The EKF linearises f by truncating the Taylor expansion after the first term:

$$f(x) \approx f(\hat{x}) + \underbrace{F_k}_{\text{1st derivative}} (x - \hat{x})$$

The next term in the series would be $\frac{1}{2}(x - \hat{x})^\top H_f (x - \hat{x})$ where H_f is the **Hessian** — the matrix of second derivatives describing the curvature of f . The EKF throws this away entirely. Any effect the curvature has on the mean or covariance is simply missed. That is what first-order accurate means: only the first derivative is used, everything else is ignored.

Why the UT is second-order accurate for any distribution.

δ is the offset from the mean to one sigma point — the direction and distance you push out. The UT evaluates f at two symmetric points: one step forward from the mean, one step backward. Taylor-expand each:

$$\begin{aligned} f(\hat{x} + \delta) &\approx f(\hat{x}) + F \cdot \delta + \frac{1}{2} \delta^\top H_f \delta \\ f(\hat{x} - \delta) &\approx f(\hat{x}) - F \cdot \delta + \frac{1}{2} \delta^\top H_f \delta \end{aligned}$$

Now add them:

$$f(\hat{x} + \delta) + f(\hat{x} - \delta) \approx 2f(\hat{x}) + \underbrace{\delta^\top H_f \delta}_{\text{2nd order term}}$$

The first-order terms $+F \cdot \delta$ and $-F \cdot \delta$ **cancel** because the points are symmetric. What survives is $2f(\hat{x})$ (twice the value at the mean) plus the curvature term $\delta^\top H_f \delta$. Dividing by 2 and weighting gives $f(\hat{x})$ plus a correction for how curved f is in direction δ — without ever computing H_f explicitly. The sigma points capture the second-order term automatically just by being symmetric. This is why the UT is **second-order accurate for any input distribution**.

Why it is third-order accurate for Gaussian inputs.

A Gaussian distribution is perfectly symmetric, so all its *odd* moments are zero — there is no skew. The third-order error term in the Taylor expansion involves the third moment of $(x - \hat{x})$. For a Gaussian that moment is exactly zero, so the third-order error vanishes automatically. The UT therefore achieves **third-order accuracy** when the input is Gaussian, at no extra cost.

Method	Accuracy	Why
EKF	1st order	Uses only Jacobian (first derivative); Hessian/curvature thrown away
UKF	2nd order (any input)	Symmetric sigma points cause first-order terms to cancel; second-order curvature captured automatically
UKF	3rd order (Gaussian input)	Odd moments of a Gaussian are zero, so the third-order error term vanishes